EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis on a Cars Dataset

Cleaning, exploring and visualising 15,411 used-car listings from India

Student name	Pallavi
Batch	October A
Submission date	30 October 2026
Project type	Individual project - Unlox Student Project Assignment
Dataset source	CarDekho used-car listings (India), public dataset: github.com/manishkr1754/CarDekho_Used_Car_Price_Prediction (file notebooks/data/cardekho_dataset.csv). Original data collected from CarDekho.com.
Tools used	Python, Pandas, NumPy, Matplotlib, Seaborn

This report contains the full analysis, charts, findings and (in the appendix) the complete Python code.

1. Objective and Research Questions

Purpose. Exploratory Data Analysis (EDA) means looking carefully at a dataset *before* building any model: checking its quality, cleaning it, and drawing charts to discover patterns. The goal of this project is to clean a used-car dataset, understand what it contains, and explain what affects the selling price of a car. No machine-learning model is built, as the assignment states.

Questions selected for analysis (all chosen because the dataset has the needed columns):

- Which brands and fuel types are the most common?
- How are car prices distributed?
- How does vehicle age relate to price?
- Do kilometres driven (and fuel mileage) appear to influence price?
- Do automatic and manual cars differ in price?
- Which brands have the highest average price?
- Which features are strongly correlated with each other and with price?
- Where are the important outliers or unusual records?

In simple words: we want to know *what makes a used car cost more or less*, and we want to check that our data is trustworthy before we say anything.

2. Dataset Overview

Source: the CarDekho used-car dataset (listings from CarDekho.com, an Indian car marketplace), downloaded from the public GitHub repository *manishkr1754/CarDekho_Used_Car_Price_Prediction*. **Size on loading:** 15,411 rows and 14 columns (13 after dropping an unnamed row-number column that carries no information). Each row is one used car listing. The dataset is large enough for meaningful analysis. It covers 31 brands and 120 car models.

Feature descriptions and data types

Column	Type	Meaning
car_name	text	Brand and model name together (e.g. Maruti Alto)
brand	text	Manufacturer
model	text	Car model
vehicle_age	integer	Age of the car in years
km_driven	integer	Total kilometres on the odometer
seller_type	text	Dealer, Individual or Trustmark Dealer
fuel_type	text	Petrol, Diesel, CNG, LPG or Electric
transmission_type	text	Manual or Automatic
mileage	decimal	Fuel efficiency (km per litre; for CNG/LPG km per kg). **Not** the odometer reading.
engine	integer	Engine size in cc
max_power	decimal	Maximum engine power in bhp
seats	integer	Number of seats
selling_price	integer	Listed selling price in Indian rupees (Rs) - target of interest

Numerical variables (7): vehicle_age, km_driven, mileage, engine, max_power, seats, selling_price. **Categorical variables (6):** car_name, brand, model, seller_type, fuel_type, transmission_type.

Sample records (first 5 rows of the raw data)

car_name	age	km_driven	seller	fuel	transm.	mileage	engine	power	seats	price (Rs)
Maruti Alto	9	120,000	Individual	Petrol	Manual	19.7	796	46.3	5	120,000
Hyundai Grand	5	20,000	Individual	Petrol	Manual	18.9	1197	82.0	5	550,000
Hyundai i20	11	60,000	Individual	Petrol	Manual	17.0	1197	80.0	5	215,000
Maruti Alto	9	37,000	Individual	Petrol	Manual	20.92	998	67.1	5	226,000
Ford Ecosport	6	30,000	Dealer	Diesel	Manual	22.77	1498	98.59	5	570,000

(The brand and model columns are shown inside car_name in this table to save space.)

3. Data Quality and Cleaning

Every check below was run on the raw data first, then a cleaning decision was made and explained.

Check	Result found	Decision
Missing values	0 missing values in all 13 columns	Nothing to fill or drop.
Duplicate rows	167 rows were exact copies of another row (same car, km, price and every other column)	Removed, keeping the first copy. Identical km and price together make a repeated listing very likely.
Inconsistent categories	Brand 'Isuzu' was written two ways: 'ISUZU' (2 rows) and 'Isuzu' (8 rows). Other text columns had no extra spaces or case differences.	Standardised to 'Isuzu', otherwise one brand would be counted as two.
Impossible values	2 cars with 0 seats; 2 cars with more than 1,000,000 km (a 5-year-old Mahindra XUV500 with 3,800,000 km and a 9-year-old Mercedes-Benz E-Class with 1,325,000 km)	Removed these 4 rows: 0 seats is impossible and the odometer readings are almost certainly typing errors.
Data types	All columns already had suitable types (numbers as numbers, text as text).	No conversion needed. Two helper columns were added: price in lakh (price / 100,000) and log10 of price.
Outliers (IQR rule)	Many statistical outliers: price 1,386, engine 2,130, max_power 867, km_driven 466	**Kept.** They are mostly genuine luxury or large-engine cars (e.g. Ferrari, Rolls-Royce, Bentley), not errors. Instead of deleting them, a log scale is used for price charts.

Dataset size before and after cleaning

Stage	Rows	Columns
Raw data after dropping the unnamed index column	15,411	13
After removing duplicates	15,244	13
After removing impossible records (final clean data)	15,240	15 (includes 2 helper columns)

In total 171 rows (1.11% of the data) were removed. The 'before' and 'after' tables are identical in structure, so the cleaning did not change the nature of the data.

In simple words: the data was already quite tidy. We removed repeated rows, fixed one spelling difference, and removed 4 clearly wrong records. We did *not* delete expensive cars, because a Ferrari costs a lot in real life - it is not a mistake.

4. Descriptive Analysis

Numerical variables (clean data, 15,240 rows)

Column	Mean	Median	Std. dev.	Min	Max	Skewness
vehicle_age	6.0	6.0	3.0	0.0	29.0	0.83
km_driven	55,315	50,000	40,675	100	950,000	5.04
mileage	19.70	19.67	4.17	4.00	33.54	0.11
engine	1,486	1,248	520	793	6,592	1.67
max_power	100.6	88.5	42.9	38.4	626.0	2.49
seats	5.3	5.0	0.8	2.0	9.0	2.09
selling_price	774,653	559,000	894,753	40,000	39,500,000	10.11

Key observations. The average selling price is Rs 774,653 but the median is only Rs 559,000. The big gap and the very high skewness (10.1) show that a small number of very expensive cars pull the average up, so the **median is a fairer 'typical price'**. A typical car is 6 years old with 50,000 km driven, a 1,248 cc engine and 5 seats. Fuel mileage is fairly symmetric (skewness 0.11) with a median of 19.67 km/l. km_driven is also strongly right-skewed (5.0) because a few cars have very high readings.

Categorical variables

Variable	Categories (share of listings)
brand	31 brands. Top 5: Maruti 32.4%, Hyundai 19.4%, Honda 9.7%, Mahindra 6.5%, Toyota 5.2%
fuel_type	Petrol 49.6%, Diesel 48.2%, CNG 2.0%, LPG 0.3%, Electric 0.0%
transmission_type	Manual 79.3%, Automatic 20.7%
seller_type	Dealer 62.0%, Individual 36.8%, Trustmark Dealer 1.1%
seats	5 seats: 12,766, 7 seats: 1,901, 8 seats: 310, 6 seats: 125, 4 seats: 76, 9 seats: 55, 2 seats: 7

Key observations. Maruti alone makes up 32.4% of listings. Petrol and diesel together cover 97.7% of cars, and manual cars are 79.3% of the data. Electric cars have only 4 listings and LPG only 44, so conclusions about those fuel types would not be reliable.

Median selling price by group

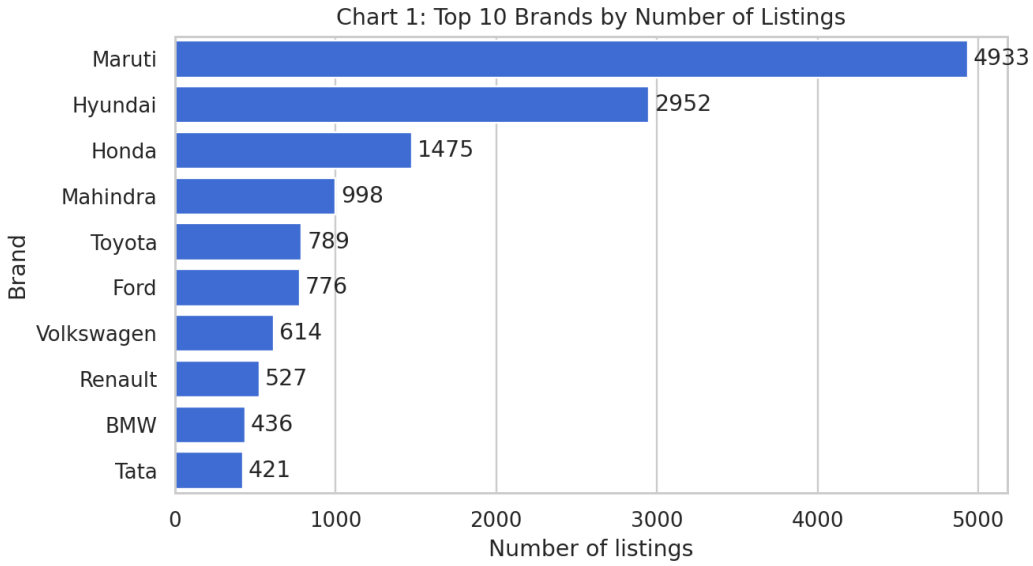
Group	Median price	Group	Median price
Manual	Rs 500,000	Petrol	Rs 460,000
Automatic	Rs 1,050,000	Diesel	Rs 700,000
Dealer	Rs 590,000	CNG	Rs 370,000
Individual	Rs 510,000	LPG (44 cars)	Rs 182,500
Trustmark Dealer	Rs 540,000	Electric (4 cars)	Rs 1,857,500

Median price by vehicle age group

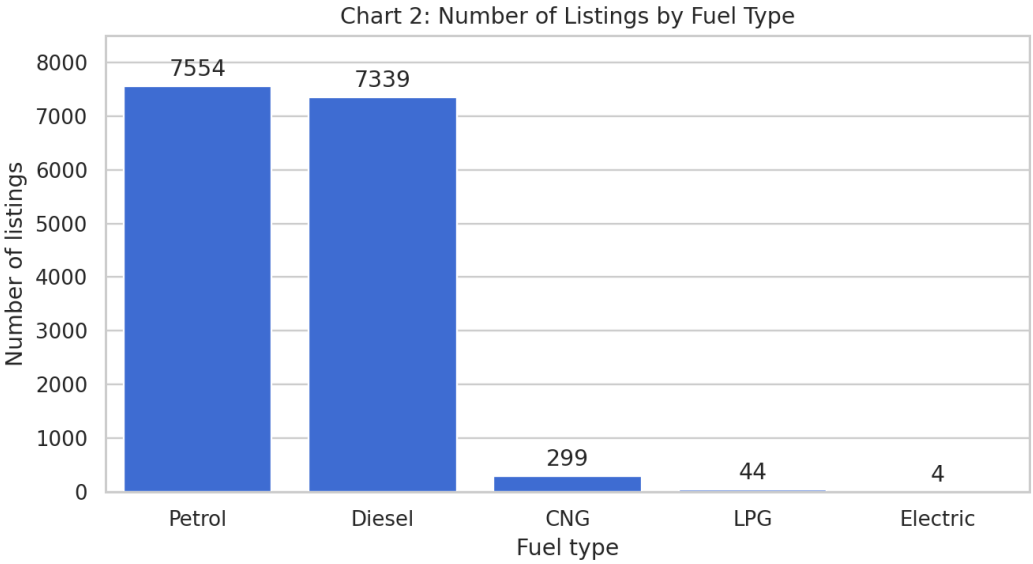
Age group	Number of cars	Median price
0-2 yrs	1,359	Rs 800,000
3-5 yrs	6,209	Rs 675,000
6-8 yrs	4,602	Rs 501,000
9-12 yrs	2,554	Rs 341,000
13+ yrs	516	Rs 180,000

5. Visual EDA

Eight charts are shown using five chart types: bar/count plots (charts 1, 2, 7), a histogram (chart 3), box plots (chart 4), scatter plots (charts 5, 6) and a heatmap (chart 8). Price axes use a log scale where noted, so that cheap and very expensive cars can be seen on the same chart. All prices are in lakh rupees (1 lakh = 100,000 Rs).

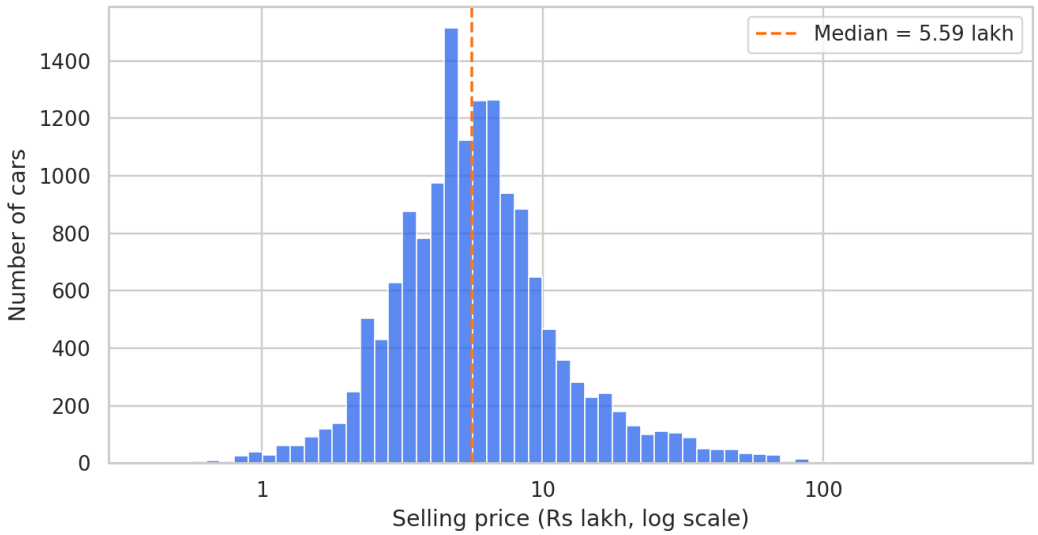


Observation: Maruti has 4,933 listings (32.4% of the data), followed by Hyundai (2,952) and Honda (1,475). These top 3 brands account for 61.4% of all listings. The market in this dataset is dominated by a few mass-market brands.



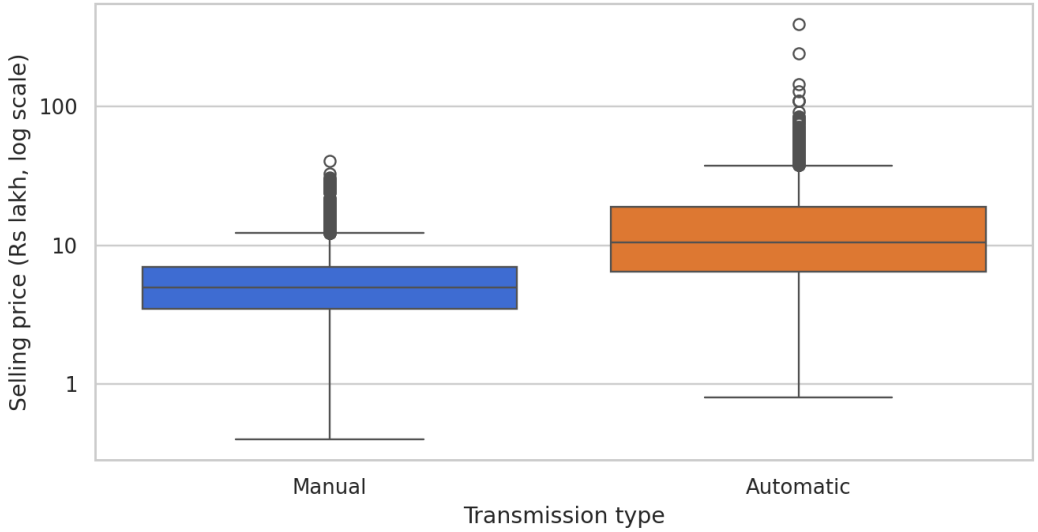
Observation: Petrol (7,554) and diesel (7,339) are almost equal and together make 97.7% of listings. CNG (299), LPG (44) and electric (4) are very rare.

Chart 3: Distribution of Selling Price (log scale)



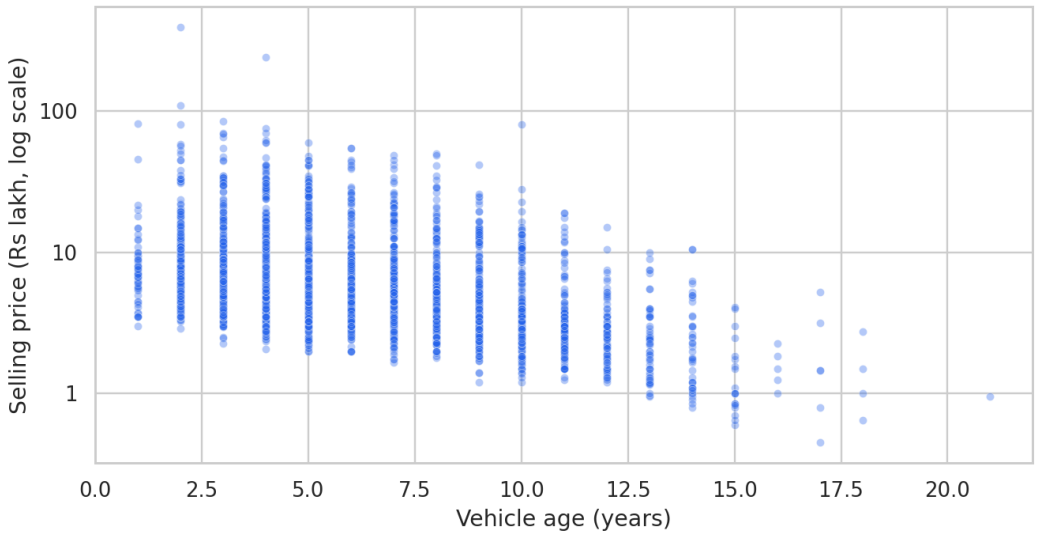
Observation: On a log scale the prices form a bell shape centred near the median of 5.59 lakh. 83.7% of cars cost Rs 10 lakh or less, but a long tail of luxury cars reaches up to 395.0 lakh. This long tail is why the mean (7.75 lakh) is much higher than the median.

Chart 4: Selling Price by Transmission Type (log scale)



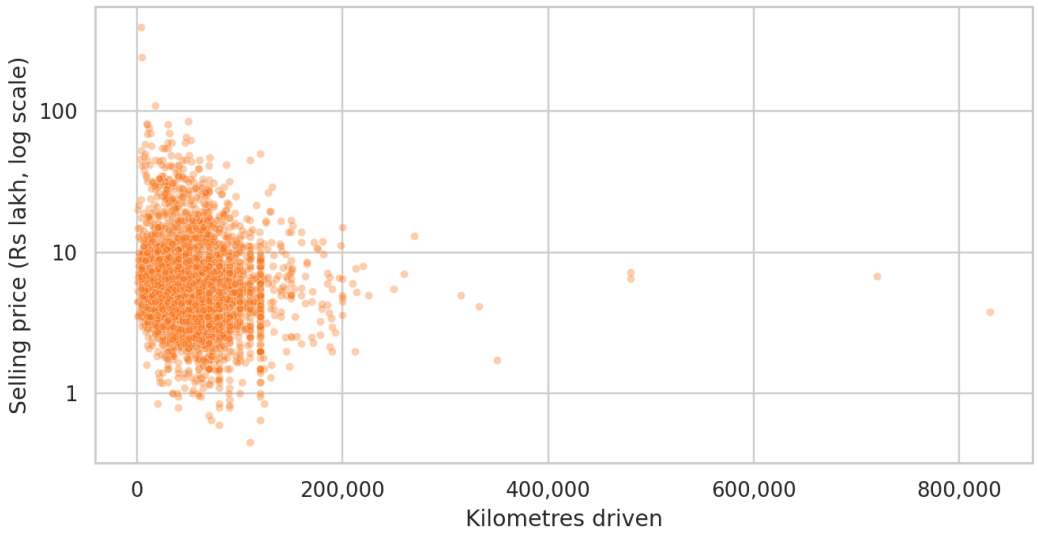
Observation: The median price of an automatic car (10.5 lakh) is about 2.1 times that of a manual car (5.0 lakh), and the whole automatic box sits higher. The two groups overlap, so some manual cars cost more than some automatics. Automatics also tend to have bigger engines (see finding 6), so transmission is not the only reason for the gap.

Chart 5: Vehicle Age vs Selling Price (random sample of 4,000)

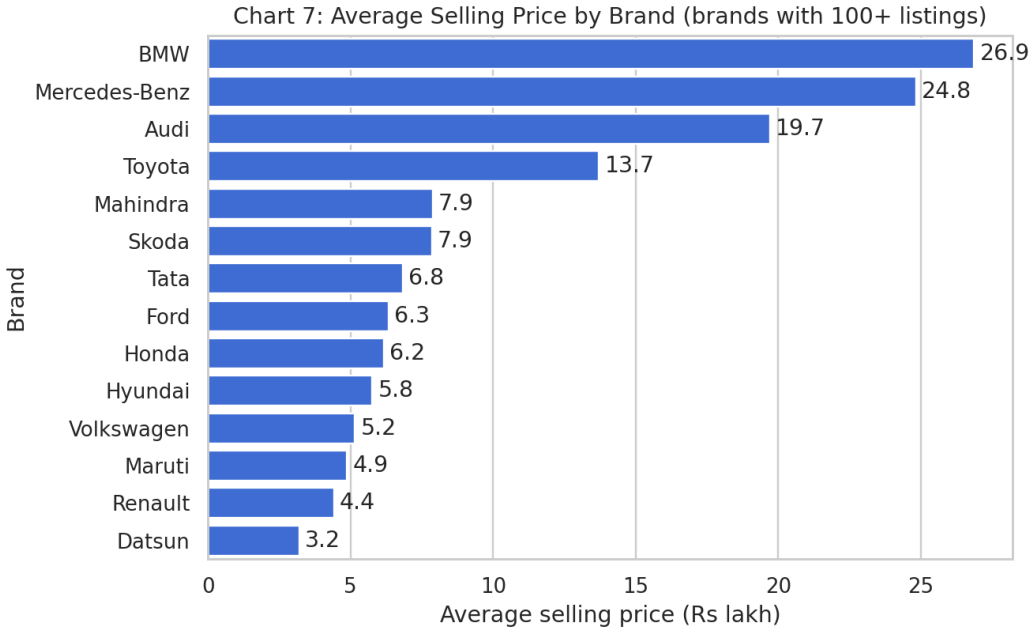


Observation: Price tends to fall as cars get older: the median price is 8.0 lakh for cars aged 0-2 years and 1.8 lakh for cars aged 13+ years. However the spread at any one age is huge: among 2,220 four-year-old cars, prices range from 2.0 lakh to 242.0 lakh. Age matters, but brand and engine matter too. (Age is in whole years, so points form vertical stripes.)

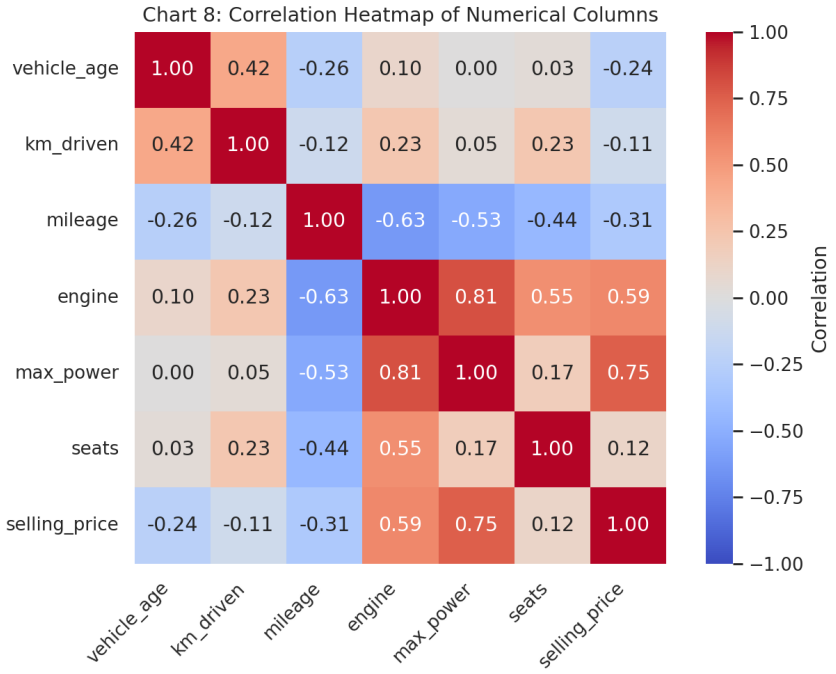
Chart 6: Kilometres Driven vs Selling Price (random sample of 4,000)



Observation: 91.7% of cars have 100,000 km or less. The cloud slopes only slightly downward: the correlation between km driven and price is just -0.11. Distance driven has a much weaker link with price than age or engine power. A few cars with very high readings are visible on the right but they do not form a clear pattern.



Observation: Among brands with at least 100 listings, BMW has the highest average price (26.9 lakh), then Mercedes-Benz (24.8 lakh) and Audi (19.7 lakh). Maruti, the most common brand, averages 4.9 lakh and Datsun is lowest at 3.2 lakh. BMW, Mercedes-Benz and Audi average roughly 4 to 5.5 times the price of Maruti.



Observation: Max power has the strongest link with selling price (r = 0.75), then engine size (r = 0.59). Vehicle age (-0.24) and km driven (-0.11) are negatively related to price. Engine and max power are strongly related to each other (r = 0.81), and engine size and fuel mileage move in opposite directions (r = -0.63).

6. Correlation Analysis

A **correlation** (r) is a number between -1 and +1 that shows how strongly two numeric columns move together. Near 0 means little relationship; near +1 or -1 means a strong one. Correlation does **not** prove one thing causes the other.

Pair	Correlation (r)	Meaning
max_power vs selling_price	0.751	Strong positive: more powerful cars cost more
engine vs selling_price	0.586	Moderate to strong positive
vehicle_age vs selling_price	-0.241	Weak-moderate negative (rank-based Spearman: -0.478)
mileage (fuel efficiency) vs selling_price	-0.305	Weak-moderate negative: efficient cars are usually small, cheaper cars
km_driven vs selling_price	-0.106	Very weak negative (Spearman: -0.166)
engine vs max_power	0.807	Very strong: they carry almost the same information
engine vs mileage	-0.632	Strong negative: bigger engines burn more fuel
vehicle_age vs km_driven	0.424	Moderate positive: older cars have driven more

Because prices are very skewed, correlations were also checked against the log of the price: max_power 0.76 and vehicle_age -0.49. The relationship with age is stronger on that scale, which agrees with the Spearman value above.

7. Key Findings

Every number below comes from the calculations in the analysis code (final clean data of 15,240 rows).

1. **A few brands dominate the market.** Maruti has 32.4% of all listings and Maruti, Hyundai and Honda together have 61.4% (Chart 1).
2. **Petrol and diesel dominate, and most cars are manual.** Petrol + diesel = 97.7% of listings; manual cars are 79.3% (Chart 2).
3. **Prices are heavily right-skewed.** The mean price (7.75 lakh) is 1.4 times the median (5.59 lakh), with skewness 10.1. The most expensive listing is a Ferrari GTC4Lusso at 395.0 lakh (Chart 3).
4. **Older cars sell for less.** Median price falls from 8.0 lakh (0-2 years) to 1.8 lakh (13+ years); the Spearman correlation of price with age is -0.48 (Chart 5).
5. **Kilometres driven has only a weak link with price.** r = -0.11 (Spearman -0.17). Since age and km are themselves related (r = 0.42), much of the km effect is already explained by age (Chart 6).
6. **Automatic cars cost about twice as much as manual cars, but they are also bigger.** Median price is 10.5 lakh vs 5.0 lakh. Median engine size is 1,798 cc vs 1,248 cc and median power is 140 bhp vs 83 bhp, so the price gap reflects car size and power as well as the gearbox (Chart 4).
7. **Engine power is the strongest numeric link with price.** r = 0.75 for max_power and 0.59 for engine size. The two are strongly related to each other (r = 0.81) (Chart 8).
8. **Luxury brands carry a large price premium.** Average prices: BMW 26.9 lakh, Mercedes-Benz 24.8 lakh, Audi 19.7 lakh versus Maruti 4.9 lakh (Chart 7). Among all brands with at least 10 listings, Porsche is highest (51.6 lakh, only 21 listings).
9. **More fuel-efficient cars are cheaper on average.** Fuel mileage has r = -0.30 with price, and bigger engines have lower mileage (r = -0.63), so efficiency here mostly reflects small, low-priced cars (Chart 8).

Unusual records (outliers)

The five most expensive listings are: Ferrari GTC4Lusso (2 yrs, 395.0 lakh); Rolls-Royce Ghost (4 yrs, 242.0 lakh); Bentley Continental (9 yrs, 145.0 lakh); Mercedes-Benz S-Class (3 yrs, 130.0 lakh); Porsche Cayenne (4 yrs, 111.0 lakh). These are real luxury cars with low kilometres, so they were kept.

Limitations of the dataset

- **Listing prices, not final sale prices.** The 'selling_price' is what the seller asked; the actual deal price may differ.

- **Association is not causation.** For example, automatic cars cost more partly because they are bigger and more powerful. This analysis cannot separate these effects.
- **Small groups.** Only 4 electric and 44 LPG cars, so fuel-type conclusions for them are unreliable.
- **Missing information.** No city/region, trim level, accident history, number of previous owners or listing date, all of which affect price.
- **One snapshot.** The data comes from a single marketplace (CarDekho) at one point in time, so it may not represent the whole Indian used-car market.
- **Cleaning rules involve judgement.** The 1,000,000 km limit and the removal of repeated rows are reasoned choices, not certainties.
- **Pre-processed copy.** The file used is a shared, already-processed version of the CarDekho data; I did not have access to the original scraping process.

8. Conclusion

The cleaned dataset of 15,240 used-car listings shows a market dominated by a few mass-market brands, mostly petrol or diesel manual cars. Selling price is highly skewed, so the median is the better measure of a typical price. Price is mainly linked to **engine power, vehicle age and brand**, while kilometres driven has a much weaker link. Automatic cars and luxury brands are more expensive, but part of this difference comes from bigger engines and higher power rather than the gearbox or badge alone.

What I learned. How to check data quality (missing values, duplicates, wrong labels, impossible values), how to decide whether an outlier is an error or a real record, why median and log scales help with skewed data, and how to read correlations without confusing them with causes.

Challenges. Deciding what to do with the extreme prices and odometer readings, and keeping every claim within what the data can support.

Possible future analysis. Compare prices within the same model and age to separate brand effects from size effects; study depreciation per model; add region or owner count if a richer dataset is available; and, if allowed in a later project, build a price-prediction model using the features found to matter here.

9. References and Dataset Source

- CarDekho used-car dataset (listings from CarDekho.com, India). GitHub repository: manishkr1754/CarDekho_Used_Car_Price_Prediction, file notebooks/data/cardekho_dataset.csv - https://github.com/manishkr1754/CarDekho_Used_Car_Price_Prediction
- CarDekho website: https://www.cardekho.com (original source of the listings).
- pandas documentation: https://pandas.pydata.org/docs/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/stable/
- Seaborn documentation: https://seaborn.pydata.org/
- Unlox Student Project Assignment: 'Exploratory Data Analysis on a Cars Dataset' (task description and requirements).

The analysis was written and run with the help of an AI assistant (Claude, by Anthropic); the code and results are reproduced in full in the appendix so they can be re-run and explained.

Appendix A: How to Explain This Project to Your Mentor

A simple script in everyday language, with the real numbers from this analysis.

1. What did you do? I took about 15,000 used-car listings from CarDekho, checked and cleaned the data, and used charts and numbers to find what affects a car's price.

2. Which dataset and why? The CarDekho dataset, because it has brand, age, km, fuel, transmission, engine, power and price - everything the assignment lists - and 15,411 rows is large enough.

3. What was wrong with the data? No missing values. But 167 rows were duplicates, 'Isuzu' was spelled two ways, and 4 rows were impossible (0 seats, or over a million km). I removed or fixed them, leaving 15,240 rows.

4. Why did you keep the very expensive cars? A Ferrari or Rolls-Royce really is that expensive. Deleting them would remove real information. I used a log scale so they do not squash the chart.

5. Why median instead of mean? A few luxury cars pull the mean up to 7.75 lakh; the median (5.59 lakh) is what a typical car costs.

6. Biggest findings? (a) Maruti is 32% of listings. (b) Older cars cost less. (c) Engine power has the strongest link with price (r = 0.75). (d) Automatics cost about twice as much as manuals, but they have bigger engines. (e) Kilometres driven matters little once age is considered.

7. Does automatic transmission make a car cost more? The data shows automatics cost more, but I cannot say the gearbox *causes* it, because automatics here also have bigger engines and more power.

8. What is a correlation? A number from -1 to +1 showing how strongly two columns move together. It does not prove one causes the other.

9. What are the limitations? Listing prices (not final prices), no location or owner details, a single website at one time, and very few electric/LPG cars.

10. What would you do next? Compare cars of the same model and age, add more features, and later build a prediction model.

How to run the code: install pandas, numpy, matplotlib and seaborn (pip install pandas numpy matplotlib seaborn), copy the code from Appendix B into a file such as Pallavi_Cars_EDA_Project.py, and run it. It downloads the dataset from the GitHub link, creates a 'charts' folder with all 8 charts, and writes the calculated numbers to results.json.

Appendix B: Complete Python Code

The code below is the complete analysis, organised in numbered sections. Lines starting with # are comments that explain each step in simple language. All numbers, tables and charts in this report were produced by this code.

```
# ======================================================================
# EXPLORATORY DATA ANALYSIS ON A CARS DATASET  (Pallavi)
# Dataset : CarDekho used-car listings (India), 15,411 rows
# Tools   : Python, Pandas, NumPy, Matplotlib, Seaborn
# ======================================================================

# ----------------------------------------------------------------------
# SECTION 1 - IMPORT LIBRARIES AND SETTINGS
# ----------------------------------------------------------------------
import json
import os
import numpy as np                 # numbers and maths
import pandas as pd                # tables (DataFrames)
import matplotlib
matplotlib.use("Agg")              # draw charts without opening a window
import matplotlib.pyplot as plt    # charts
import seaborn as sns              # nicer statistical charts

sns.set_theme(style="whitegrid", context="notebook")
OUT_DIR = "charts"                 # folder where chart images are saved
os.makedirs(OUT_DIR, exist_ok=True)
results = {}                       # we store every calculated number here


def plain_numbers(axis):
    """Show 1, 10, 100 on a log axis instead of 10^0, 10^1, 10^2."""
    axis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:g}"))


def save_chart(filename):
    """Tidy the layout and save the current chart as a PNG image."""
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, filename), dpi=170)
    plt.close()


# ----------------------------------------------------------------------
# SECTION 2 - LOAD THE DATA
# ----------------------------------------------------------------------
DATA_URL = ("https://raw.githubusercontent.com/manishkr1754/"
            "CarDekho_Used_Car_Price_Prediction/main/notebooks/data/"
            "cardekho_dataset.csv")
try:
    raw = pd.read_csv(DATA_URL)
except Exception:                  # fallback if there is no internet
    raw = pd.read_csv("cardekho.csv")

# ----------------------------------------------------------------------
# SECTION 3 - FIRST LOOK: SHAPE, COLUMNS, DATA TYPES, SAMPLE ROWS
# ----------------------------------------------------------------------
# The first column "Unnamed: 0" is only an old row number, so we drop it.
df = raw.drop(columns=["Unnamed: 0"]).copy()
rows_before, cols_before = df.shape
results["shape_before"] = [rows_before, cols_before]
results["dtypes"] = {c: str(t) for c, t in df.dtypes.items()}
results["sample_rows"] = df.head(5).values.tolist()
results["columns"] = list(df.columns)

# Split columns into numerical and categorical groups
num_cols = df.select_dtypes(include="number").columns.tolist()
cat_cols = df.select_dtypes(exclude="number").columns.tolist()
results["num_cols"] = num_cols
results["cat_cols"] = cat_cols

# ----------------------------------------------------------------------
# SECTION 4 - DATA QUALITY CHECKS (BEFORE CLEANING)
# ----------------------------------------------------------------------
results["missing_total"] = int(df.isna().sum().sum())
results["missing_by_col"] = df.isna().sum().to_dict()

# Duplicates: the same car, same km, same price, same everything.
results["duplicates"] = int(df.duplicated().sum())

# Inconsistent labels: spelling/case variants of the same brand name.
```

```
lower_names = df["brand"].str.lower()
label_counts = df.groupby([lower_names, df["brand"]]).size()
results["brand_label_issues"] = {
    name: label_counts[name].to_dict()
    for name in lower_names.unique()
    if len(label_counts[name]) > 1}      # same brand written 2+ ways

# Impossible values (domain knowledge): a car cannot have 0 seats;
# a car with over 1,000,000 km is not believable for these records.
results["rows_zero_seats"] = int((df["seats"] == 0).sum())
results["rows_km_over_1m"] = int((df["km_driven"] > 1_000_000).sum())
results["removed_km_rows"] = df.loc[df["km_driven"] > 1_000_000,
                                    ["car_name", "vehicle_age",
                                     "km_driven"]].values.tolist()

# Outliers by the IQR rule (before treatment) - for information only
def iqr_outliers(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return int(((series < q1 - 1.5 * iqr) | (series > q3 + 1.5 * iqr)).sum())

results["iqr_outliers_before"] = {c: iqr_outliers(df[c]) for c in
                                  ["km_driven", "mileage", "engine",
                                   "max_power", "selling_price"]}

# ---------------------------------------------------------------------
# SECTION 5 - DATA CLEANING
# ---------------------------------------------------------------------
# 5.1 Remove leading/trailing spaces in text columns
for c in cat_cols:
    df[c] = df[c].str.strip()

# 5.2 Fix inconsistent brand label: "ISUZU" and "Isuzu" -> "Isuzu"
df["brand"] = df["brand"].replace({"ISUZU": "Isuzu"})
df["car_name"] = df["car_name"].str.replace("ISUZU", "Isuzu", regex=False)

# 5.3 Remove duplicate rows (keep the first copy)
df = df.drop_duplicates().reset_index(drop=True)
rows_after_dup = len(df)

# 5.4 Remove impossible records
df = df[df["seats"] > 0]                 # 0 seats is impossible
df = df[df["km_driven"] <= 1_000_000]    # implausible odometer readings
df = df.reset_index(drop=True)

# 5.5 Outlier policy: luxury cars (Ferrari, Rolls-Royce...) are REAL, so
# we keep them. We use a log scale for price instead of deleting them.
df["price_lakh"] = df["selling_price"] / 100_000      # price in lakh Rs
df["log_price"] = np.log10(df["selling_price"])

rows_after, cols_after = df.shape
results["shape_after"] = [rows_after, cols_after]
results["rows_after_dup_removal"] = rows_after_dup
results["rows_removed_total"] = rows_before - rows_after
results["iqr_outliers_after"] = {c: iqr_outliers(df[c]) for c in
                                 ["km_driven", "mileage", "engine",
                                  "max_power", "selling_price"]}

# ---------------------------------------------------------------------
# SECTION 6 - DESCRIPTIVE STATISTICS
# ---------------------------------------------------------------------
stat_cols = ["vehicle_age", "km_driven", "mileage", "engine",
             "max_power", "seats", "selling_price"]
desc = df[stat_cols].describe().T
desc["median"] = df[stat_cols].median()
desc["skew"] = df[stat_cols].skew()
results["describe"] = desc.round(2).reset_index().rename(
    columns={"index": "column"}).to_dict(orient="records")

# Categorical summaries
for c in ["fuel_type", "transmission_type", "seller_type"]:
    results["share_" + c] = (df[c].value_counts(normalize=True) * 100
                             ).round(2).to_dict()
    results["count_" + c] = df[c].value_counts().to_dict()
results["brand_counts"] = df["brand"].value_counts().to_dict()
results["n_brands"] = int(df["brand"].nunique())
results["n_models"] = int(df["model"].nunique())
results["seats_counts"] = df["seats"].value_counts().to_dict()

# ---------------------------------------------------------------------
```

```
# SECTION 7 - BIVARIATE / MULTIVARIATE NUMBERS
# ---------------------------------------------------------------------
# Median price by transmission, fuel and seller type
for c in ["transmission_type", "fuel_type", "seller_type"]:
    results["median_price_" + c] = (df.groupby(c)["selling_price"]
                                    .median().round(0).to_dict())
results["median_engine_by_transmission"] = (
    df.groupby("transmission_type")["engine"].median().to_dict())
results["median_power_by_transmission"] = (
    df.groupby("transmission_type")["max_power"].median().to_dict())

# Brands with at least 100 listings -> average and median price
brand_n = df["brand"].value_counts()
big_brands = brand_n[brand_n >= 100].index
brand_price = (df[df["brand"].isin(big_brands)]
               .groupby("brand")["selling_price"]
               .agg(["count", "mean", "median"]).sort_values("mean",
                                                             ascending=False))
results["brand_price_table"] = brand_price.round(0).reset_index().to_dict(
    orient="records")

# Brands (all) with highest average price, min 10 listings
bp_all = (df.groupby("brand")["selling_price"].agg(["count", "mean"])
          .query("count >= 10").sort_values("mean", ascending=False).head(5))
results["top5_expensive_brands_min10"] = bp_all.round(0).reset_index(
).to_dict(orient="records")

# Median price by vehicle age group
df["age_group"] = pd.cut(df["vehicle_age"], bins=[-1, 2, 5, 8, 12, 40],
                         labels=["0-2 yrs", "3-5 yrs", "6-8 yrs",
                                 "9-12 yrs", "13+ yrs"])
results["median_price_by_age_group"] = (
    df.groupby("age_group", observed=True)["selling_price"]
    .agg(["count", "median"]).round(0).reset_index().astype(
        {"age_group": str}).to_dict(orient="records"))

# Correlations (Pearson) and rank correlations (Spearman)
corr_cols = ["vehicle_age", "km_driven", "mileage", "engine",
             "max_power", "seats", "selling_price"]
corr = df[corr_cols].corr()
results["corr_matrix"] = corr.round(3).to_dict()
results["corr_with_price"] = corr["selling_price"].drop(
    "selling_price").round(3).to_dict()
results["spearman_price_age"] = round(
    df["selling_price"].corr(df["vehicle_age"], method="spearman"), 3)
results["spearman_price_km"] = round(
    df["selling_price"].corr(df["km_driven"], method="spearman"), 3)
results["pearson_logprice_age"] = round(
    df["log_price"].corr(df["vehicle_age"]), 3)
results["pearson_logprice_power"] = round(
    df["log_price"].corr(df["max_power"]), 3)
results["corr_age_km"] = round(df["vehicle_age"].corr(df["km_driven"]), 3)
results["corr_engine_power"] = round(df["engine"].corr(df["max_power"]), 3)
results["corr_engine_mileage"] = round(df["engine"].corr(df["mileage"]), 3)
results["corr_power_mileage"] = round(df["max_power"].corr(df["mileage"]), 3)


# A few extra facts used to explain the charts
age4 = df.loc[df["vehicle_age"] == 4, "selling_price"]
results["age4_min_max"] = [int(age4.min()), int(age4.max())]
results["age4_count"] = int(len(age4))
results["share_km_le_100k"] = round((df["km_driven"] <= 100_000).mean() * 100, 1)
results["share_price_le_10lakh"] = round(
    (df["selling_price"] <= 1_000_000).mean() * 100, 1)
results["share_age_le_8"] = round((df["vehicle_age"] <= 8).mean() * 100, 1)
results["km_p99"] = float(df["km_driven"].quantile(0.99))

# Top-5 most expensive records (outlier inspection)
results["top5_price_records"] = (
    df.nlargest(5, "selling_price")[["car_name", "vehicle_age",
                                     "km_driven", "fuel_type",
                                     "selling_price"]].values.tolist())

# ---------------------------------------------------------------------
# SECTION 8 - VISUALISATIONS (8 charts, 5 chart types)
# ---------------------------------------------------------------------
BLUE, ORANGE = "#2563eb", "#f97316"

# Chart 1 - Bar/count plot: top 10 brands by number of listings
top_brands = df["brand"].value_counts().head(10)
```

```
plt.figure(figsize=(8, 4.5))
ax = sns.barplot(x=top_brands.values, y=top_brands.index, color=BLUE)
ax.bar_label(ax.containers[0], padding=3)
plt.title("Chart 1: Top 10 Brands by Number of Listings")
plt.xlabel("Number of listings")
plt.ylabel("Brand")
save_chart("c1_top_brands.png")

# Chart 2 - Bar/count plot: fuel type
order = df["fuel_type"].value_counts().index
plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=df, x="fuel_type", order=order, color=BLUE)
ax.bar_label(ax.containers[0], padding=3)
ax.set_ylim(0, 8500)
plt.title("Chart 2: Number of Listings by Fuel Type")
plt.xlabel("Fuel type")
plt.ylabel("Number of listings")
save_chart("c2_fuel_type.png")

# Chart 3 - Histogram: price distribution (log scale on x-axis)
plt.figure(figsize=(8, 4.5))
sns.histplot(df["price_lakh"], bins=60, log_scale=True, color=BLUE)
med = df["price_lakh"].median()
plain_numbers(plt.gca().xaxis)
plt.axvline(med, color=ORANGE, ls="--", label=f"Median = {med:.2f} lakh")
plt.title("Chart 3: Distribution of Selling Price (log scale)")
plt.xlabel("Selling price (Rs lakh, log scale)")
plt.ylabel("Number of cars")
plt.legend()
save_chart("c3_price_hist.png")

# Chart 4 - Box plot: price by transmission type
plt.figure(figsize=(8, 4.5))
sns.boxplot(data=df, x="transmission_type", y="price_lakh",
            palette=[BLUE, ORANGE], hue="transmission_type", legend=False)
plt.yscale("log")
plain_numbers(plt.gca().yaxis)
plt.title("Chart 4: Selling Price by Transmission Type (log scale)")
plt.xlabel("Transmission type")
plt.ylabel("Selling price (Rs lakh, log scale)")
save_chart("c4_price_transmission.png")

# Chart 5 - Scatter plot: vehicle age vs price
plt.figure(figsize=(8, 4.5))
sns.scatterplot(data=df.sample(4000, random_state=42), x="vehicle_age",
                y="price_lakh", alpha=0.35, s=18, color=BLUE)
plt.yscale("log")
plain_numbers(plt.gca().yaxis)
plt.title("Chart 5: Vehicle Age vs Selling Price (random sample of 4,000)")
plt.xlabel("Vehicle age (years)")
plt.ylabel("Selling price (Rs lakh, log scale)")
save_chart("c5_age_price.png")

# Chart 6 - Scatter plot: km driven vs price
plt.figure(figsize=(8, 4.5))
sns.scatterplot(data=df.sample(4000, random_state=42), x="km_driven",
                y="price_lakh", alpha=0.35, s=18, color=ORANGE)
plt.yscale("log")
plain_numbers(plt.gca().yaxis)
plt.gca().xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:,.0f}"))
plt.title("Chart 6: Kilometres Driven vs Selling Price (random sample of 4,000)")
plt.xlabel("Kilometres driven")
plt.ylabel("Selling price (Rs lakh, log scale)")
save_chart("c6_km_price.png")

# Chart 7 - Bar plot: average price of brands with >= 100 listings
bp = brand_price["mean"].sort_values(ascending=False) / 100_000
plt.figure(figsize=(8, 5))
ax = sns.barplot(x=bp.values, y=bp.index, color=BLUE)
ax.bar_label(ax.containers[0], fmt="%.1f", padding=3)
plt.title("Chart 7: Average Selling Price by Brand (brands with 100+ listings)")
plt.xlabel("Average selling price (Rs lakh)")
plt.ylabel("Brand")
save_chart("c7_brand_price.png")

# Chart 8 - Heatmap: correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1,
            square=True, cbar_kws={"label": "Correlation"})
plt.title("Chart 8: Correlation Heatmap of Numerical Columns")
```

```
plt.xticks(rotation=45, ha="right")
save_chart("c8_corr_heatmap.png")

# Extra: price by fuel type (numbers only, used for findings)
results["price_by_fuel_describe"] = (df.groupby("fuel_type")["selling_price"]
                                     .agg(["count", "median", "mean"])
                                     .round(0).reset_index().to_dict(
                                         orient="records"))

# ---------------------------------------------------------------------
# SECTION 9 - SAVE RESULTS (used to build the report)
# ---------------------------------------------------------------------
with open("results.json", "w") as f:
    json.dump(results, f, indent=1, default=str)
print("Analysis finished. Charts saved in:", OUT_DIR)
```